

EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

Ghidra Reversing Tutorial: Static Analysis of the Stripped RP2350 Image

ARM Cortex-M33 • GDB • Ghidra

October 1, 2026

For educational and defensive security research only

0. Cold Open

Open the image and it will lie to you with total confidence.

That is not a bug. That is the tool doing exactly what it was built to do: propose. Ghidra proposes functions. It proposes boundaries. It proposes names for things it cannot possibly know. And when it is wrong it will not tell you, because it cannot tell the difference between a guess and a fact any better than the machine that answered nine seconds too fast.

A stripped binary is a wall of Thumb-2 with the labels torn off. The only way through is patience, cross-references, and the refusal to accept a story just because it is plausible. Ghidra is the map. It is not the territory.

So you draw the map, then you walk the ground. You separate the two coordinate pairs by what *references* them, not by what they look like. You find the eight bytes that decide where a machine goes, and you prove which eight they are.

The tool proposes. The bench disposes.

Think, then verify.

1. Executive Summary

This tutorial reverses the **stripped** `0x0011a_cb.bin` with Ghidra: no symbols, no sections, no metadata. We import the raw image, recover the code graph, correct the boundaries Ghidra gets wrong, find the hardcoded target waypoint, and patch it. Every output below is from the raw image at base `0x10000000`, language `ARM:LE:32:Cortex`.

2. Import the Raw Image

The `.bin` is a raw XIP image. You must tell Ghidra where it lives and how to decode it.

GHIDRA IMPORT SETTINGS	
Language :	ARMvLE:32:Cortex (ARMv8-M / Cortex-M33, Thumb-2)
Format :	Raw Binary
Base :	0x10000000 (RP2350 external flash / XIP)

2.1 Headless (reproducible), per host

macOS:

```
GHIDRA=/Applications/ghidra_12.0.4_PUBLIC
"$GHIDRA/support/analyzeHeadless" /tmp/ghproj DarkVector \
  -import 0x0011a_cb.bin \
  -processor "ARM:LE:32:Cortex" \
  -loader BinaryLoader -loader-baseAddr 0x10000000
```

Linux:

```
GHIDRA=$HOME/ghidra_12.0.4_PUBLIC
"$GHIDRA/support/analyzeHeadless" /tmp/ghproj DarkVector \
  -import 0x0011a_cb.bin \
  -processor "ARM:LE:32:Cortex" \
  -loader BinaryLoader -loader-baseAddr 0x10000000
```

Windows PowerShell:

```
$GHIDRA = "C:\ghidra_12.0.4_PUBLIC"
& "$GHIDRA\support\analyzeHeadless.bat" "%env:TEMP\ghproj" DarkVector `
    -import 0x0011a_cb.bin `
    -processor "ARM:LE:32:Cortex" `
    -loader BinaryLoader -loader-baseAddr 0x10000000
```

2.2 GUI, per host

Launch Ghidra, then **File -> Import File**, set language and base address, and analyze:

- macOS: /Applications/ghidra_12.0.4_PUBLIC/ghidraRun
- Linux: \$HOME/ghidra_12.0.4_PUBLIC/ghidraRun
- Windows: C:\ghidra 12.0.4 PUBLIC\ghidraRun.bat

3. Entry and the Stripped-Binary Problem

The first two words are the ARMv8-M vector table:

```
0x10000000: 0x20082000 ; initial SP
0x10000004: 0x1000015D ; reset handler (Thumb)
```

EMBEDDED HACKING

Ghidra recovers **166 functions** from the call graph. But with no symbols it gets some boundaries wrong, and it folds tail-call-only helpers into their callers: `release_payload` is reached only by a `b.w` tail call from `navigate_to_target` at `0x10000D18`, so Ghidra does not give it its own function. It names the entry `FUN_10000234`. The disassembly is right; the boundaries are not. Correcting them is the analyst's job, and only the bench confirms them.

Applying correct boundaries yields:

Address	Function
0x10000234	main
0x10000300	init_gps_pio
0x1000040C	poll_gps
0x10000858	gps_get_stats
0x10000870	init_lora
0x10000A08	lora_send
0x10000A54	lora_tick
0x10000AC4	init_propeller
0x10000AF8	propeller_set_bearing
0x10000B34	propeller_stop
0x10000B64	init_payload
0x10000BB0	set_gnss_leds
0x10000BFC	release_payload
0x10000C2C	init_navigation
0x10000C7C	send_telemetry
0x10000CB8	navigate_to_target
0x10000E5C	aes128_ecb_decrypt_block
0x100012D4	init_lcd
0x1000175C	lcd_show_coords
0x10001B4C	lcd_show_gnss

4. `main` as Ghidra Sees It (Raw Image)

```
void main(void)
{
    local_18 = *DAT_100002f4; // ORIGIN_LAT @ 0x1000A098
    uStack_14 = DAT_100002f4[1];
    local_10 = *DAT_100002f8; // ORIGIN_LON @ 0x1000A090
    uStack_c = DAT_100002f8[1];
    FUN_10005d98(); // stdio_init_all
    FUN_10000c2c(); // init_navigation
    FUN_10000b64(); // init_payload
    FUN_10000870(); // init_lora
    FUN_10000300(); // init_gps_pio
    FUN_10000ac4(); // init_propeller
    FUN_100012d4(); // init_lcd
    piVar1 = DAT_100002fc; // &hold @ 0x200010F0
    do {
        while( true ) {
            iVar3 = 200;
            bVar4 = 0;
            local_20 = 0;
            uStack_1c = 0;
            do {
                bVar2 = FUN_1000040c(&local_18,&local_10); // poll_gps
                bVar4 = bVar2 | bVar4;
                FUN_10000a54(); // lora_tick
                FUN_10002ab8(5); // sleep_ms(5)
                iVar3 = iVar3 + -1;
            } while ( iVar3 != 0 );
            FUN_10000858(&local_20,&uStack_1c); // gps_get_stats
            if (bVar4 == 0) break;
            *piVar1 = 3; // hold = 3
            FUN_10000bb0(1,local_20); // set_gnss_leds
LAB_100002a4:
            FUN_1000175c(local_18,uStack_14,local_10,uStack_c); // lcd_show_coords
            FUN_10000cb8(local_18,uStack_14,local_10,uStack_c); // navigate_to_target
        }
        iVar3 = *piVar1;
        if (iVar3 < 1) { iVar3 = 1; }
        *piVar1 = iVar3 + -1; // hold--
        if (iVar3 + -1 != 0) {
            FUN_10000bb0(1,local_20); // set_gnss_leds
            goto LAB_100002a4;
        }
        FUN_10000bb0(0,local_20); // set_gnss_leds(0,...)
        FUN_10001b4c(local_20,uStack_1c); // lcd_show_gnss
        FUN_10000b34(); // propeller_stop
        FUN_10000c7c(local_18,uStack_14,local_10,uStack_c); // send_telemetry
    } while( true );
}
```

`FUN_` prefixes everywhere: this is what a stripped target really looks like.

5. The Decoy And The Encrypted Target

`navigate_to_target` no longer compares against a literal; it reads the pointers at `DAT_10000e50` / `DAT_10000e54`, which resolve to RAM `0x20000D00` / `0x20000D08`, **runtime doubles**. Where do they come from? `init_navigation`, and that is the whole puzzle.

```
void init_navigation(void)
{
    local_28 = *DAT_10000c6c; // key @ 0x10009D90
    uStack_24 = DAT_10000c6c[1];
    uStack_20 = DAT_10000c6c[2];
    uStack_1c = DAT_10000c6c[3];
    local_18 = *DAT_10000c70; // ct @ 0x10009DA4
    uStack_14 = DAT_10000c70[1];
    uStack_10 = DAT_10000c70[2];
    uStack_c = DAT_10000c70[3];
    FUN_10000e5c(&local_18,&local_28,&local_38); // aes128_ecb_decrypt_block
    *DAT_10000c74 = local_38; // TARGET_LAT -> 0x20000D00
    puVar1[1] = uStack_34;
    *puVar2 = local_30; // TARGET_LON -> 0x20000D08
    puVar2[1] = uStack_2c;
}
```

Three data addresses do all the work:

Symbol	Address	Meaning
CTF_AES_KEY	0x10009D90	the AES key 564543544f5231314145534b45592121
CTF_TARGET_CT	0x10009DA4	the encrypted target pair
TARGET_LAT/LON	0x20000D00 / 0x20000D08	RAM, reconstructed at boot

Reading the key bytes (Ghidra will call them code):

`0x10009D90` is **data**, not code. `init_navigation` copies the block into a stack buffer and hands it to `aes128_ecb_decrypt_block`; it is never executed. Ghidra still marks it as code because it sees the read cross-reference from `init_navigation` (`FUN_10000c2c:10000c36(R)`) and guesses. The Listing then shows fabricated mnemonics:

```

LAB_10009d90      XREF[1]: FUN_10000c2c:10000c36(R)
10009d90 56 45      cmp      r6,r10
10009d92 43 54      strb     r3,[r0,r1]
10009d94 4f 52      strh     r7,[r1,r1]
10009d96 31 31      adds     r1,#0x31
10009d98 41 45      cmp      r1,r8
10009d9a 53 4b      ldr      r3,[s_n,"%s"_failed:_file_"%s",_line = "n \"%s\" failed: file
10009d9c 45 59      ldr      r5,[r0,r5]
10009d9e 21 21      movs     r1,#0x21

```

Those are the key bytes, not instructions. Two traps:

1. The mnemonics are meaningless: 56 45 is V,E; 43 54 is C,T; 4f 52 is 0,R.
2. At 0x10009d9a the halfword 53 4b decodes as ldr r3, [pc, #332], whose literal-pool target is 0x10009EE8. That address really does hold a string, newlib's assert message Assertion "%s" failed: file "%s", line %d%s at 0x10009EE0, so Ghidra prints its label. The string is real, but the reference is spurious: 53 4b is S,K, bytes 10-11 of the key, and only decodes as that ldr by coincidence. The same thing happens at 0x10009DA4, where the ciphertext byte pair 20 4f decodes to an ldr that lands on the real two-space string at 0x10009E28.

Read the bytes, never the mnemonics. Any of these is exact:

```
xxd -s 0x9D90 -l 16 0x0011a_cb.bin      # raw image, no tools
```

```
(gdb) x/16bx 0x10009D90                # live target
```

In the Ghidra GUI the simplest path is **Window -> Bytes**, press **G**, enter 0x10009D90, and read the 16 raw bytes. To retype them in the Listing instead, select the 16 bytes, press **C** (Clear Code/Data), then **T** (Define Data) and choose **byte**; press **I** to make it an array of 16. Note that **B** is **not** "define byte" in Ghidra, it is *Cycle Integer Types*, and data cannot be defined over bytes that are still typed as code, which is why the clear (**C**) must come first. The value is the ASCII key VECTOR11AESKEY!! : 56 45 43 54 4F 52 31 31 41 45 53 4B 45 59 21 21.

Decrypt the target (fully offline, reproducible):

Step 1. The key, 16 bytes at 0x10009D90 (read them as above):

```

56 45 43 54 4F 52 31 31 41 45 53 4B 45 59 21 21  = "VECTOR11AESKEY!!"
hex for tools: 564543544f5231314145534b45592121

```

Step 2. The ciphertext, 16 bytes at 0x10009DA4. Read them the exact same way (**Window -> Bytes**, press **G**, enter 0x10009DA4):

```

20 4F AC C2 C0 23 EA 7A 3A 93 0F 71 11 23 EF CA
hex for tools: 204facc2c023ea7a3a930f711123efca

```

Step 3. AES-128-ECB decrypt the block with the key (no padding). The plaintext is two little-endian IEEE-754 doubles, latitude then longitude.

Python, all platforms (cryptography is already used by the course):

```

import struct
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes

key = bytes.fromhex("564543544f5231314145534b45592121")
ct = bytes.fromhex("204facc2c023ea7a3a930f711123efca")

pt = Cipher(algorithms.AES(key), modes.ECB()).decryptor().update(ct)
print("plaintext:", pt.hex())
print("lat:", struct.unpack("<d", pt[:8])[0])
print("lon:", struct.unpack("<d", pt[8:])[0])

```

Output:

```

plaintext: 6284f068e3704340bf823463d15c53c0
lat: 38.88194
lon: -77.45028

```

<d is a little-endian 64-bit double, exactly how the firmware reads the pair.

Same result with openssl (macOS, Linux, WSL):

```

printf '\x20\x4f\xac\xc2\xc0\x23\xea\x7a\x3a\x93\x0f\x71\x11\x23\xef\xca' \
| openssl enc -aes-128-ecb -K 564543544f5231314145534b45592121 -nopad -d \
| xxd
# 00000000: 6284 f068 e370 4340 bf82 3463 d15c 53c0

```

Same result in CyberChef (browser, nothing to install):

1. **From Hex** on 204facc2c023ea7a3a930f711123efca.
2. **AES Decrypt**: Key = Hex 564543544f5231314145534b45592121, Mode = ECB, Padding = None, Input/Output = Raw.
3. **To Hex** to read 6284f068e3704340bf823463d15c53c0.
4. That is two 8-byte little-endian doubles: 62 84 F0 68 E3 70 43 40 and BF 82 34 63 D1 5C 53 C0.

Step 4. Decode the two doubles with the bit layout

$$V = (-1)^s \times 2^{e-1023} \times (1 + m)$$

EMBEDDED HACKING

The decrypted bytes are little-endian, so reverse each 8-byte group to the big-endian hex word the Week 5 utility expects:

```
python3 scripts/float_hex_converter.py 0x404370E368F08462 # +38.881940
python3 scripts/float_hex_converter.py 0xC0535CD1633482BF # -77.450280
```

`struct.unpack("<d", ...)` already performs that byte swap for you.

Crypto boundary (design note). The target is a single 16-byte AES-128-ECB block under a hardcoded ASCII key. That is intentionally minimal and weak: ECB mode, one block, and the key sitting in the image, so anyone holding the `.bin` can decrypt it. It is chosen to make the offline decrypt fit a lesson, not as a model of sound cryptography. Treat it as a puzzle, not a recipe.

And the plaintext pair at `0x1000A090 / 0x1000A098` (`+38.840280 / -77.428890`)? That is the **decoy**, the beacon's broadcast, and it is a lie. The real target only exists after the AES.

6. release_payload (Raw Decompilation)

```
void release_payload(void)
{
    coprocessor_moveto2(0,4,0x10,1,in_cr0); // GPIO16 = 1
    coprocessor_moveto2(0,4,0x11,1,in_cr0); // GPIO17 = 1
    coprocessor_moveto2(0,4,0x12,1,in_cr0); // GPIO18 = 1
    __wrap_puts(uRam10009d3c); // "PAYLOAD RELEASED AT TARGET COORDINATES"
    lora_send(uRam10009d64); // tail call
}
```

`coprocessor_moveto2` is Ghidra's rendering of the RP2350 GPIO `mccr p0, #4` path. A model will "helpfully" tell you this is a normal SIO store; the encoding says otherwise, and GDB confirms it live.

Two coordinate pairs (the decoy)

The image contains **two** hardcoded double pairs, not one:

Pair	Address	Meaning
Launch origin (Centreville)	<code>0x1000A098 / 0x1000A090</code>	broadcast by the crash beacon
Target (NRO HQ)	<code>0x20000D00 / 0x20000D08</code>	RAM, rebuilt by <code>init_navigation</code>

A pattern-matcher latches onto the **broadcast** origin and calls it the target. The only way to tell them apart is the cross-reference: the target pair is loaded with `ldr` and fed to `__aeabi_dcmpeq` inside `navigate_to_target`; the origin pair is handed to `send_telemetry`. Strings and intuition are not enough.

7. The Patch: Re-vector to a Safe Waypoint

The target is the 16 bytes at `0x10009DA4`. To re-vector the drone you replace that block with the AES-128-ECB encryption of a safe waypoint (`37.0 / -74.0`, open Atlantic) under the same key. Compute the safe doubles and their ciphertext:

```
python3 scripts/float_hex_converter.py 37.0 # 0x4042800000000000
python3 scripts/float_hex_converter.py -74.0 # 0xC052800000000000

python3 -c "import struct,sys; sys.stdout.buffer.write(struct.pack('<d',37.0)+struct.pack('<d',-74.0))" \
| openssl enc -aes-128-ecb -K 564543544f5231314145534b45592121 -nopad | xxd -p
# c2bb647c8778bf59279c01ad066bb16b
```

In the Ghidra Listing, press **G** to `0x10009DA4`, select the 16 bytes, then **Ctrl+Shift+G** (Patch Data):

Address	Original	Patched
<code>0x10009DA4</code>	<code>20 4F AC C2 C0 23 EA 7A 3A 93 0F 71 11 23 EF CA</code>	<code>C2 BB 64 7C 87 78 BF 59 27 9C 01 AD 06 6B B1 6B</code>

Export: **File -> Export Program...** -> Format **Binary** -> `0x0011a_cb_patched.bin`.

8. Export, Convert, Flash, Verify

```
python3 uf2conv.py 0x0011a_cb_patched.bin \
-f 0xe48bff59 -b 0x10000000 -c -o 0x0011a_cb_patched.uf2
```

Hold **BOOTSEL**, copy the UF2 across, and verify against the live ground HUD. A patch that is not flashed and confirmed is a guess.

9. Randomized Builds

Each student image embeds a unique key and waypoint, so no answer key travels:

```
python3 scripts/randomize_build.py --student-id alice --seed 12345 --uf2
[+] student_id : alice
[+] TARGET_LAT : 38.880272
[+] TARGET_LON : -77.460077
[+] AES key : <16 random bytes>
[+] ciphertext : <AES-128-ECB(key, target)>
[+] image : build-ctf/0x0011a_cb_alice.uf2
[+] answer key : <keydir>/answer_alice.json (INSTRUCTOR ONLY, do not ship)
```

Addresses are identical; only the key and ciphertext bytes differ.

10. Reproducibility Checklist

1. Import raw `.bin`, ARM:LE:32:Cortex, base `0x10000000`.
2. Reset vector `[0]=0x20082000`, `[1]=0x1000015D`.
3. `main @ 0x10000234`; fix the merged boundary at a single `FUN_` function.
4. `navigate_to_target @ 0x10000CB8` reads the pointers at `0x10000E50 / 0x10000E54`, targeting RAM `0x20000D00 / 0x20000D08`.
5. Decode to `+38.881940 / -77.450280`.
6. Patch, export, `uf2conv`, flash, verify.

11. Why Buddy Fails Here

Ghidra itself proves the point: it *proposes* functions and boundaries, and the analyst corrects them with cross-references and the bench. A language model does the same, faster and wrong, with no way to confirm. It cannot validate a boundary, cannot read the live `ldrd`, and cannot flash a patch to see the drone re-vector. Structure is a hypothesis; silicon is the verdict. **Think, then verify.**

Appendix A. Deep Ghidra Step-Through

A.1 Import

Language ARM:LE:32:Cortex, Base Address `0x10000000`, Raw Binary. Analyze.

A.2 Address map

Address	What it is
<code>0x10000234</code>	<code>main</code>
<code>0x10000C2C</code>	<code>init_navigation</code> , rebuilds the target
<code>0x10000E5C</code>	<code>aes128_ecb_decrypt_block</code>
<code>0x10000CB8</code>	<code>navigate_to_target</code> , the arrival compare
<code>0x10009D90</code>	AES key
<code>0x10009DA4</code>	ciphertext
<code>0x1000A090</code>	decoy waypoint

A.3 The S-box

Go to `0x1000A1AC` and find the 256-byte S-box that starts `63 7C 77 7B` (the inverse S-box, starting `52 09 6A D5`, sits at `0x1000A0AC`). Right click, create an array of 256 bytes.

A.4 Recover

Take 16 bytes at `0x10009DA4` and the key at `0x10009D90`, then run the Python decrypt block from section 5.

A.5 Patch

Overwrite the 16 bytes at `0x10009DA4` with the re-encrypted Atlantic block, then File, Export Program, Format Binary.

A.6 Platform note

Ghidra is identical on Windows, Linux, and macOS. Only the paths differ.

Action	macOS	Linux	Windows
GUI launch	<code>/Applications/ghidra_12.0.4_PUBLIC/ghidraRun</code>	<code>\$HOME/ghidra_12.0.4_PUBLIC/ghidraRun</code>	<code>C:\ghidra_12.0.4_PUBLIC\ghidraRun</code>
Headless	<code>/Applications/ghidra_12.0.4_PUBLIC/support/analyzeHeadless</code>	<code>\$HOME/ghidra_12.0.4_PUBLIC/support/analyzeHeadless</code>	<code>C:\ghidra_12.0.4_PUBLIC\suppo</code>
Shell	<code>zsh / bash</code>	<code>bash</code>	PowerShell

See sections 2.1 and 2.2 for the full import commands.